

Machine Learning Course

Dr. Serin Atiani

AGResU-Net for BraTS MRI Brain Tumor Segmentation

Phase 3 Technical Report

Team Members: Layan Abandeh | Mira Diab | Sewar Abdelrahman

Date: 30 / 4 / 2026

1. Introduction

In this report, we describe our work for Phase 3 of the AGResU-Net project, which focuses on automatic MRI brain tumor segmentation using the BraTS 2017, BraTS 2018, and BraTS 2019 datasets.

What we built is a complete TensorFlow/Keras training pipeline that can be run locally in VS Code once the BraTS datasets are downloaded. The pipeline covers everything from GPU setup and automatic patient-folder discovery to 2D patch extraction, spatial augmentation, the AGResU-Net model itself, Dice-based metrics, a combined loss function, K-fold validation, training plots, and fold summaries.

2. Dataset and Local Setup

We downloaded all three datasets locally using Kaggle commands and extracted them under the project data folder. Each dataset is placed in a dedicated subfolder:

Dataset	Local Folder Used by Our Code
BraTS 2017	PROJECT_DIR / data / brats17
BraTS 2018	PROJECT_DIR / data / brats2018
BraTS 2019	PROJECT_DIR / data / brats2019

For a patient folder to be considered valid, it needs to contain all four MRI modalities (FLAIR, T1, T1ce, T2) plus the segmentation mask. Our patient discovery function scans the extracted folders recursively and only keeps folders that have all the required files.

3. Methodology

3.1 Preprocessing

We load each patient as a 4-channel MRI volume using nibabel, with channels ordered as FLAIR, T1, T1ce, and T2. Each modality is normalized separately using z-score normalization after clipping extreme intensity values at the 1st and 99th percentiles. This is consistent with what the paper describes regarding intensity normalization and handling MRI variation.

The segmentation mask uses BraTS labels 0, 1, 2, and 4. Since TensorFlow's softmax expects contiguous class indices, we remap label 4 to class 3. This gives us four classes: background, necrosis/non-enhancing tumor, edema, and enhancing tumor.

We convert the 3D volumes into axial 2D slices and prioritize slices that contain tumor pixels. Each slice is center-cropped or padded to 128×128 pixels, giving a model input of $128 \times 128 \times 4$, which matches what the paper uses.

Item	Our Implementation
Input shape	$128 \times 128 \times 4$
Output shape	$128 \times 128 \times 4$ softmax classes
MRI modalities	FLAIR, T1, T1ce, T2
Mask labels	0, 1, 2, 4 — label 4 remapped to 3 internally

3.2 Data Augmentation

We added three more augmentation types. Every augmentation is applied to both the image and its mask simultaneously. For MRI images, we use bilinear interpolation, and for masks, we use nearest-neighbor to preserve label values.

Augmentation	Why We Use It
Horizontal and vertical flips	Increases orientation variation across training samples.
Random rotation (-10° to $+10^\circ$)	Helps the model be robust to small spatial differences in tumor position.
Random shift (up to ~ 8 pixels)	Improves robustness to variation in where the tumor appears within the slice.
Gaussian noise	Simulates MRI intensity noise and helps reduce overfitting.

3.3 AGResU-Net Architecture

Our model is a 2D AGResU-Net. It combines residual blocks in the encoder and decoder with attention gates in the skip connections, and produces a single softmax output at the end. We removed deep supervision because the paper's AGResU-Net architecture does not use it.

Stage	Operation	Filters	Spatial Size
Input	MRI stack	4 channels	128×128
Encoder 1	Residual block	64	128×128
Encoder 2	Downsample residual block	128	64×64
Encoder 3	Downsample residual block	256	32×32
Bridge	Downsample residual block	512	16×16
Decoder 3	Upsample + attention gate + residual block	256	32×32

Decoder 2	Upsample + attention gate + residual block	128	64×64
Decoder 1	Upsample + attention gate + residual block	64	128×128
Output	Conv 1×1 + Softmax	4 classes	128×128

Residual block: Each residual block applies Batch Normalization, PReLU, and 3×3 convolution. When the number of channels or spatial size changes, we use a 1×1 projection shortcut.

Attention gate: The attention gate compares the skip feature $\mathbf{x}$ with the gating feature $\mathbf{g}$. It creates an attention map α , then multiplies it by $\mathbf{x}$ to keep important features and reduce irrelevant ones.

3.4 Loss Function and Metrics

We use a combined loss: Generalized Dice Loss plus Weighted Cross Entropy. This is consistent with the paper, which used a union loss of these two components to deal with the heavy class imbalance that is typical in BraTS segmentation.

Component	Role in Our Code
Generalized Dice Loss	Improves learning under class imbalance.
Weighted Cross Entropy	Gives more weight to tumor classes relative to background.
Optimizer	SGD
Learning rate	0.086
Momentum	0.97
Evaluation metrics	Dice WT, Dice TC, Dice ET

3.5 Training Procedure

We train separately for each dataset using K-fold validation. We recommend running the code with EPOCHS = 2 first to verify that the folder setup, generator, GPU, and architecture are all working before moving to a full training run.

Setting	Test Run	Full Training Run
Datasets	BraTS 2017 first	BraTS 2017, 2018, and 2019
Epochs	2	80
Batch size	4, depending on GPU memory	4 or 8, depending on GPU memory
K-folds	5 in code (can reduce for testing)	5
Slices per patient	40 for quick run	60 or more for full training
Checkpoint	Best validation Dice WT	Best validation Dice WT

4. Results and Comparison with the Paper

The paper reports AGResU-Net performance using Dice Similarity Coefficient (DSC) for three tumor regions: Whole Tumor (WT), Tumor Core (TC), and Enhancing Tumor (ET). The table below shows the paper values we are using as our benchmark.

Dataset / Experiment	Paper WT	Paper TC	Paper ET
BraTS 2017 training split	0.876	0.772	0.720
BraTS 2017 validation	0.880	0.781	0.749
BraTS 2018 validation	0.872	0.808	0.772

Our current run was a short pipeline-verification run, not a full training run. The goal was simply to confirm that everything works end-to-end on our local GPU and that the plots and fold summaries are generated correctly. From that short run, the Dice values were approximately:

Run Type	WT Dice	TC Dice	ET Dice	Explanation
Our short local test run	0.042	0.035	0.004	Very low — the model ran for only 2 epochs and had not converged.
Paper — BraTS 2017 validation	0.880	0.781	0.749	Full training with the complete dataset and paper setup.
Paper — BraTS 2018 validation	0.872	0.808	0.772	Full training with the complete dataset and paper setup.

4.1 Why Our Results Are Lower Than the Paper

The gap between our short-run values and the paper values is fully expected. The paper was trained on the complete BraTS training sets and evaluated using a proper multi-fold protocol. Our short run was only meant to verify that the pipeline runs without errors — not to reproduce the final numbers.

Reason for Lower Current Results	Effect on Dice
Very few epochs in the test run	The model mostly predicts background or incomplete tumor regions.
Limited slices and patients in the quick test	The validation set may not represent the full range of tumor variation.
No full K-fold result yet	The current result is not an averaged performance number across folds.
2D implementation limitation	The paper also notes that 2D models lose some 3D context between slices.
GPU memory constraints	Batch size and slices per patient may need tuning based on available memory.

To get results that can be fairly compared with the paper, we would need to run the full experiment: 5-fold validation, all available labeled patients, at least 80 epochs, and the saved best checkpoint for each fold. Once we do that, we will update the results table with mean WT, TC, and ET Dice scores across folds.

5. Steps We Took to Work Toward the Paper Results

Here is a summary of everything we did to bring our implementation closer to the paper:

- Downloaded BraTS 2017, 2018, and 2019 using Kaggle commands and extracted them into local folders.
- Verified the TensorFlow environment and confirmed GPU detection before any training.
- Implemented the paper input format using $128 \times 128 \times 4$ 2D MRI patches.
- Corrected the architecture so the first encoder level starts with 64 filters, not 16.
- Implemented residual blocks and attention gates in the skip connections.
- Removed deep supervision, which is not part of the paper's AGResU-Net architecture.
- Added Generalized Dice Loss combined with Weighted Cross Entropy to handle class imbalance.
- Replaced the misleading pixel accuracy metric with Dice scores for WT, TC, and ET.
- Added K-fold validation to better match the paper's evaluation protocol.
- Added training-curve and Dice-score plots for analyzing and reporting results.

6. Team Contributions

All team members worked together on all parts of the project. We prepared the environment, downloaded and organized the datasets, wrote and tested the preprocessing steps, implemented the AGResU-Net model, added the attention gates, loss function, Dice metrics, training loop, plots, and result comparison. We also reviewed the code and report together to make sure the final submission was complete and matched the paper requirements.

References

Zhang, J., Jiang, Z., Dong, J., Hou, Y., & Liu, B. (2020). Attention Gate ResU-Net for Automatic MRI Brain Tumor Segmentation. *IEEE Access*, 8, 58533–58545.

BraTS Challenge datasets: BraTS 2017, BraTS 2018, and BraTS 2019.